Real-Time Object Detection Using YOLOv8 For Smart Automation

OLMCA699 - Project

Submitted in partial fulfillment of the requirements for the degree of

Master of Computer Applications

in

Department of Computer Applications

by

MYTHREYA DUVVURI

24EMCA1319

Under the guidance of

Prof. Karthikeyan D

School of Computer Science and Engineering

VIT, Vellore



June, 2026

Real-Time Object Detection Using YOLOv8 For Smart Automation

OLMCA699 - Project

Submitted in partial fulfillment of the requirements for the degree of

Master of Computer Applications

in

Department of Computer Applications

by

MYTHREYA DUVVURI

24EMCA1319

Under the guidance of

Prof. Karthikeyan D

School of Computer Science and Engineering

VIT, Vellore



DECLARATION

I hereby declare that the **Project** entitled “Real-Time Object Detection Using YOLOv8 For Smart Automation" submitted by me, for the award of the degree of *Master of Computer Applications in Department of Computer Applications, School of Computer Science and Engineering* to VIT is a record of bonafide work carried out by me under the supervision of Prof. Karthikeyan D, Assistant Professor Sr. Grade 1, School of Computer Science and Engineering, VIT, Vellore.

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

Date: 30-06-2026

Mythreya Duvvuri
Signature of the Candidate

CERTIFICATE

This is to certify that the **Project** entitled “Real-Time Object Detection Using YOLOv8 For Smart Automation” submitted by Mythreya Duvvuri - 24EMCA1319, SCOPE, VIT, for the award of the degree of *Master of Computer Applications in Department of Computer Applications*, is a record of bonafide work carried out by him under my supervision during the period, 27/03/26 to 03/07/26, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfill the requirements and regulations of the University and in my opinion meet the necessary standards for submission.

Place: Vellore

Date:

Signature of the VIT-SCORE/SCOPE - Guide

Head of the Department

Department of Computer Applications

ACKNOWLEDGEMENT

It is my pleasure to express with a deep sense of gratitude to my **Project** guide Prof. Karthikeyan D, Assistant Professor Sr. Grade 1**,** School of Computer Science and Engineering Vellore Institute of Technology, Vellore for his constant guidance, continual encouragement, in my endeavor. My association with him is not confined to academics only, but it is a great opportunity on my part to work with an intellectual and an expert in the field of Computer Science**.**

"I would like to express my heartfelt gratitude to Honorable Chancellor **Dr. G Viswanathan**; respected Vice Presidents **Mr. Sankar Viswanathan**, **Dr. Sekar Viswanathan**, Vice Chancellor **Dr. V. S. Kanchana Bhaaskaran**; Pro-Vice Chancellor **Dr. Partha Sharathi Mallick**; and Registrar **Dr. Jayabarathi T**.

My whole-hearted thanks to Director, VITOL, School of Computer Science and Engineering Head, Department of Computer Applications, all faculty, staff and members working as limbs of our university for their continuous guidance throughout my course of study in unlimited ways.

It is indeed a pleasure to thank my parents and friends who persuaded and encouraged me to take up and complete my **project** successfully. Last, but not least, I express my gratitude and appreciation to all those who have helped me directly or indirectly towards the successful completion of the **project.**

Place: Vellore
Date: 30-06-2026

Mythreya Duvvuri

Executive Summary

Real-time object detection is crucial for modern automation. While current state-of-the-art models perform well on structured Western roads, deploying these systems in an unstructured environment such as Indian streets with high traffic density and localized vehicle types presents a unique challenge. Furthermore, to run these heavy neural networks, expensive, cloud computing hardware is required. This project evaluates the feasibility of deploying a localizable, real-time object detection pipeline on a standard, consumer-grade computer.

To address this, three sizes of the single-stage detector, YOLOv8 (Small, Large, and Extra-Large), were trained and evaluated using the Indian Driving Dataset (IDD). The objective was to identify the trade-offs between model size. This was measured in accuracy (mAP), and processing speed (latency in milliseconds) when running on local hardware.

The metrics were collected inside Jupyter Notebook using YOLOv8's native validation engine, and the results were plotted using Matplotlib. A custom inference pipeline was built using OpenCV to print the model's inference speed directly on top of the image. It was run locally on an NVIDIA RTX 3070 Laptop GPU with 8 GB of VRAM.

TABLE OF CONTENTS

CONTENTS	Page No.
ACKNOWLEDGEMENT	i
Executive Summary	ii
TABLE OF CONTENTS	iii
List of Figures	v
List of Tables	vi
List of Abbreviations	vii
Symbols and Notations	viii
1 INTRODUCTION	**1**
1.1 Objective	1
1.2 Motivation	2
1.3 Background	2
2 PROJECT DESCRIPTION AND GOALS	**3**
2.1 Evolution of Object Detection Models	3
2.2 YOLO Architectures for Real-Time Processing	3
2.3 Applications of YOLO in Edge/Automation Environments	4
2.4 Conclusion and Model Selection	4
2.5 Project Objectives	5
3 TECHNICAL SPECIFICATION	6
3.1 Dataset Selection and Sampling Plan	6
3.2 Tools and Technologies Identified	6
4 DESIGN APPROACH	**8**
4.1 Dataset Management and Preprocessing	8
4.1.1 Dataset Acquisition and Structural Analysis	8

4.1.2 Leakage Prevention and Stratified Splitting 8
4.1.3 Image Preprocessing and Formatting via Roboflow 9
4.1.4 Cloud Notebook Integration 10
4.2 Model Training Strategy and Configuration 10
4.3 Local Inference and Evaluation Setup 11
4.3.1 Hardware and Software Environment 11
4.3.2 Evaluation Process and Metrics 11
4.3.3 Core Operational Insight 12

5 SCHEDULE, TASKS AND MILESTONES 13
5.1 Project Task Breakdown 13
5.2 Project Milestones and Timeline 14

6 PROJECT DEMONSTRATION 15
6.1 Verification and Evaluation Setup 15
6.2 Custom Inference Pipeline and Visual Overlay 18

7 PERFORMANCE ANALYSIS AND DISCUSSION 23
7.1 Analysis of YOLOv8 Sizes 23
7.2 Single-Stage Architecture vs. Two-Stage 23
7.3 Domain Challenges on the IDD Dataset 23
7.4 Limitations of the Model 24

8 SUMMARY 25

9 REFERENCES
APPENDIX A
APPENDIX B
APPENDIX C

List of Figures

Figure No.	Title	Page No.
4.1	Image Resizing with Black Edges	10
6.1	Model Accuracy & Performance Metrics and Inference Speed Performance	16
6.2	Normalized Confusion Matrix of YOLOv8-Small on Test Data	16
6.3	Normalized Confusion Matrix of YOLOv8-Large on Test Data	17
6.4	Normalized Confusion Matrix of YOLOv8-Extra Large on Test Data	17
6.5	YOLOv8 Small Model Prediction-1	19
6.6	YOLOv8 Small Model Prediction-2	19
6.7	YOLOv8 Large Model Prediction-1	20
6.8	YOLOv8 Large Model Prediction-2	20
6.9	YOLOv8 Extra-Large Model Prediction-1	21
6.10	YOLOv8 Extra-Large Model Prediction-1	21

List of Tables

Table No.	Title	Page No.
4.1	Class Distribution Across Dataset Splits	9
5.1	Project Milestones with Timeline	14
6.1	Accuracy and Inference Speed Metrics by Model	15

List of Abbreviations

CNN	Convolutional Neural Network
CUDA	Compute Unified Device Architecture
CV	Computer Vision
FPS	Frames Per Second
Faster R-CNN	Faster Region-based Convolutional Neural Network
GPU	Graphics Processing Unit
I/O	Input/Output
IDD	Indian Driving Dataset
R-CNN	Region-based Convolutional Neural Network
RPN	Region Proposal Network
YOLO-v1	You Only Look Once (Version 1)
YOLOv8	You Only Look Once (Version 8)

Symbols and Notations

mAP@0.5	Mean Average Precision at an Intersection over Union (IoU) threshold of 0.5
mAP@0.5:0.95	Mean Average Precision averaged over multiple IoU thresholds from 0.5 to 0.95 with a step size of 0.05
ms	milliseconds

CHAPTER 1
INTRODUCTION

The rapid adaptation of smart automation and intelligent systems relies on the ability to process visual data in real time. For applications such as localized traffic monitoring, systems must rapidly identify diverse objects to make immediate informed decisions. Historically, these systems transmitted high-bandwidth video feeds to centralized cloud servers, introducing critical latency and bandwidth bottlenecks. As the computation for automation shifts toward edge devices, where data is processed locally at the source, there is a requirement for object detection architectures that are highly accurate yet lightweight enough to operate on consumer-grade hardware.

To achieve this, the system must adapt a continuous, high-speed computational workflow. It takes a live image or video stream as input, pre-processes the frames such as resizing and normalizing them, and then runs an inference model. During inference, the model detects target objects, assigns class labels, and generates bounding boxes. Finally, the system displays these predictions on the screen in real time. The YOLOv8 architecture is an ideal framework to handle this. By keeping this entire end-to-end workflow on edge hardware, the system presents an optimal, highly responsive solution for modern automated environments.

1.1 Objective

The primary objective of this project is to demonstrate and evaluate the feasibility of using YOLOv8 object detection models for real-time inference on a consumer-grade device equipped with a standard GPU. The goal is to prove that high-speed, accurate computer vision pipelines can be achieved locally without relying on expensive, enterprise-level cloud infrastructure. This research aims to demonstrate a practical solution for real-time smart automation.

1.2 Motivation

Traditional two-stage models such as Faster R-CNN require too much computing power to run smoothly on standard, consumer-grade hardware. The problem, therefore, is a lack of a localized, lightweight detection model that can function without relying on expensive cloud infrastructure. This project addresses this gap by fine-tuning YOLOv8 on a custom dataset, optimizing it to detect localized objects efficiently on standard GPU hardware.

1.3 Background

The foundations of smart automation relies heavily on the ability of systems to perceive, interpret, and autonomously react to their physical environments. While traditional automation systems depended heavily on hard-coded operational rules, static environments, and human involvement, modern frameworks can adapt to dynamic environments using situational awareness. This technology can integrate into pre-existing systems and improve the accuracy, speed, and reliability of the system[1]. Whether applied to industrial automation, smart city infrastructure, or robotics, systems must now process complex visual data to make instantaneous, context-aware decisions.

To achieve this, the integration of advanced Computer Vision (CV) is essential, specifically real-time object detection. The continuous evolution of artificial intelligence and the transition toward automated industrial environments has led to the widespread adoption of visual detection technologies[2]. Unlike basic image classification, which merely identifies the presence of an entity, object detection localizes multiple entities within a given frame, providing critical spatial coordinates. Without highly accurate and rapidly processing object detection, smart automation remains limited to pre-programmed routines. Therefore, developing lightweight, high-speed visual detection models is not just an enhancement for smart automation, but a foundational necessity.

CHAPTER 2
PROJECT DESCRIPTION AND GOALS

2.1 Evolution of Object Detection Models

Early object detection methods relied on handcrafted features and rigid templates. For a long time, the algorithms could only identify simple structures, such as handwritten numbers, or basic geometric shapes. The field advanced significantly with the integration of Convolutional Neural Networks (CNNs). CNNs revolutionized the landscape by automating feature extraction, allowing models to learn complex visual patterns directly from data leading to the development of deep-learning-based detection models[3].

Early deep learning architectures primarily utilized a "two-stage" approach, exemplified by the R-CNN (Region-based Convolutional Neural Network) family, which includes Fast R-CNN and Faster R-CNN. In these two-stage models, the architecture first generates a series of region proposals, guessing where an object might be; and subsequently classifies the object within those regions[4]. While two-stage detectors achieve exceptionally high precision and accuracy, their sequential processing nature creates a significant computational bottleneck. This latency renders them largely unsuitable for real-time smart automation tasks.

To resolve the latency limitations inherent in region-proposal networks, the introduction of the YOLO (You Only Look Once) framework reframed object detection as a single regression problem, simultaneously predicting bounding boxes and class probabilities in one pass[5]. By processing the entire image through the neural network in a single pass, this architecture drastically reduced inference times. This finally bridged the gap between high-level computer vision and the strict latency requirements of real-time automated systems.

2.2 YOLO Architectures for Real-Time Processing

Since its inception, the YOLO framework has undergone rapid architectural iterations to optimize the trade-off between detection accuracy and computational efficiency. The

YOLO-v1 working principle was to use a grid structure and impose it on the image. The cell containing the object of interest would then predict a bounding box as all the cells that contain the object. It used 24 convolutional layers and two fully connected layers[6]. From then, the later versions incorporated new and emerging technologies like DarkNet backbone, mosaic data augmentation, Path Aggregation Network, Dynamic Kernel Attention; as well as better optimization techniques such as Momentum and Adam optimizer, and post-training quantization[7]. These newer architectures also offer varying model sizes (ranging from Nano to Extra-Large) allowing developers to choose lightweight models that prioritize low latency for immediate actuation tasks, or heavier models where precise detection of complex or occluded objects is prioritized over absolute speed.

While each model was developed on top of the previous one, they perform differently on different tasks. A study found that YOLO v5, v7 and v8 showed inference speeds of 1.3ms, 4.93ms and 3.3ms respectively while maintaining overall mAP(Mean Average Precision)@0.5 scores of 0.94, 0.441 and 0.927 respectively[8]. While YOLO v5 scored better at overall mAP@0.5, YOLO v8 scored better at Precision.

2.3 Applications of YOLO in Edge/Automation Environments

YOLO framework's true utility is demonstrated through its deployment in dynamic, real-world smart automation systems. Smart automation requires systems to move beyond passive analysis, utilizing real-time visual data to determine responses. This requires balancing precision against inference speeds. While two-stage architectures like Faster R-CNN can achieve superior accuracy metrics for specific, complex detection tasks, they suffer from severe computational latency. One analysis demonstrated that single-stage models like YOLOv5 can achieve inference rates up to ten times faster than Faster R-CNN under identical hyperparameter conditions[9]. YOLO v5 scored mAP@0.5 of 53.05% at an inference rate of 0.017 seconds per image while Faster R-CNN scored mAP@0.5 of 63.575% at an inference rate of 0.17 seconds per image.

2.4 Conclusion and Model Selection

The existing literature establishes that while traditional two-stage architectures may offer precision benefits in highly specialized, static imaging tasks, their inherent computational

latency makes them fundamentally unsuitable for modern smart automation. As demonstrated across multiple benchmarks, the YOLO framework's ability to process visual data up to ten times faster than models like Faster R-CNN makes it the better choice for real-time applications.

Consequently, to address the strict latency and accuracy requirements of smart automation, this project will utilize the YOLO architecture, specifically YOLOv8. YOLOv8 was selected because it represents an optimal balance in the framework's evolution, utilizing its efficient anchor-free design that is more flexible without sacrificing the high-speed inference necessary for immediate system actuation. With its decoupled heads for classification and localization tasks, it is more generalizable as well as converges faster, letting us fine tune it. By implementing YOLOv8, this project aims to develop a highly responsive, robust, and deployable solution for real-time object detection in automated environments.

2.5 Project Objectives

To address the challenges posed in the problem statement, this project is guided by the following objectives:

- System Implementation: Implement and deploy a high-speed, real-time object detection system based on the YOLOv8 architecture.
- Visual Output Generation: Process video feeds and generate bounding boxes, class labels, and confidence scores, utilizing the OpenCV library.
- Custom Fine-Tuning: Adapt the architecture by fine-tuning it on a custom dataset, enabling the recognition of localized and unstructured target classes such as auto-rickshaws.
- Architectural Comparison: Compare the computational efficiency, latency, and precision of the YOLOv8 model against traditional, two-stage CNN-based detectors.
- Real-Time Demonstration: Demonstrate the real-time capabilities of the model by validating its inference speeds on local, consumer-grade GPU hardware, proving its viability for edge deployment.

CHAPTER 3
TECHNICAL SPECIFICATION

3.1 Dataset Selection and Sampling Plan

To demonstrate the adaptability of the YOLOv8 architecture for custom classes, this project utilizes a fine-tuning approach. The sampling plan involves aggregating real-world traffic imagery. The model will be explicitly trained to detect three custom classes highly relevant to South Asian traffic environments: potholes, auto-rickshaws, and two-wheeler helmets.

Data will be sourced from open-source repositories such as the Indian Driving Dataset (IDD)[10]. To facilitate supervised learning, the raw, unstructured data will be manually annotated using Roboflow tools, establishing precise bounding box coordinates for the target classes.

To ensure the model generalizes effectively and to prevent overfitting during the fine-tuning process, the final annotated dataset will be partitioned using a standard stratified split:
70% of the data allocated for model training, 20% reserved for continuous validation during training epochs, and a strict 10% hold-out testing set. This testing set will be used exclusively to evaluate the model's final inference metrics, specifically testing its Accuracy, Precision, Recall and mean Average Precision (mAP) as well as the inference times when exposed to entirely unseen scenarios.

3.2 Tools and Technologies Identified

To execute the data preparation, model fine-tuning, and evaluation phases, this project will employ a specialized computer vision technology stack. The selected tools are optimized for handling deep learning architectures and real-time processing:

- Algorithm Framework: Ultralytics YOLOv8. Selected for its anchor-free architecture, providing an optimal balance between mean Average Precision (mAP) and real-time inference speeds (Frames Per Second). Three variants namely small, large and extra large are selected for their varying sizes and performances.

- Programming Language & Core Libraries: Python, supported by the PyTorch deep learning backend. PyTorch will handle tensor computations and neural network graphing, while OpenCV (Open Source Computer Vision Library) will be utilized during the testing phase to process video feeds and render bounding boxes.

- Data Annotation & Pipeline Management: Roboflow. This platform will serve as the primary tool for manual bounding-box annotation, dataset augmentation, and exporting the final dataset into the required YOLOv8 format.

- Training Environment & Hardware: Kaggle, Google Colab. Due to the high computational requirements of training Convolutional Neural Networks (CNNs), cloud-based environments equipped with dedicated GPUs will be utilized to accelerate training epochs and manage large batch sizes effectively. Jupyter Notebook for local inference, benchmarking and collecting metrics.

- Real-Time Inference & Edge Deployment: To empirically validate the model's real-time capabilities, post-training inference testing will be conducted locally on a consumer-grade system equipped with a dedicated CUDA-enabled GPU. This phase will utilize OpenCV to process live video feeds and pre-recorded footage.

- Performance Visualization: Matplotlib will be used to visualize training metrics, generating loss curves and Precision-Recall graphs to analytically present the model's progress.

CHAPTER 4
DESIGN APPROACH

4.1 Dataset Management and Preprocessing

4.1.1 Dataset Acquisition and Structural Analysis

The project utilizes the Indian Driving Dataset (IDD) Detection dataset[10] to train and evaluate the models under real-world unstructured traffic scenarios. Upon initial investigation, the raw IDD data was found to be organized chronologically in nested folder structures, containing a varying number of images per folder. The bounding box annotation was in Pascal VOC format.

4.1.2 Leakage Prevention and Stratified Splitting

Because the data consists of continuous sequences, a random split of individual images would cause data leakage. Meaning nearly identical frames from the same sequence would end up in both the training and testing sets, artificially inflating the model's accuracy. To prevent this, the data split was done at the folder level. Entire sequences were kept together and manually divided into separate training, validation, and testing sets, aiming as closely as possible for a standard 70% / 20% / 10% ratio with even class distribution. This ensures the model is evaluated on completely new driving environments.

The final dataset contained 10239 images for training, 2744 for validation and 1410 for testing. The following classes with their number in their respective splits were chosen to train and evaluate the models.

Table 4.1 Class Distribution Across Dataset Splits

Class label	Count in Train	Count in Valid	Count in Test
autorickshaw	22690	6175	3415
bicycle	1000	258	162
bus	7110	1682	931
car	41347	8832	4109
motorcycle	45485	11078	6558
person	39633	8950	4256
rider	45473	10694	6417
truck	11292	2469	1538

4.1.3 Image Preprocessing and Formatting via Roboflow

Once the data was split, the images needed to be standardized for the model. Since the original IDD images have their camera's resolution dimensions, the dataset was uploaded to Roboflow to handle the preprocessing steps:

- Resizing to Standard Dimensions: All images were resized to a uniform 640 x 640 pixels, which is the native input size for YOLOv8.
- Aspect Ratio Preservation: To stop the images from stretching or distorting, which makes it harder for a neural network to recognize objects, black edges were added to fill the empty space while keeping the original proportions of the camera frame.



Fig. 4.1 Image Resizing with Black Edges

4.1.4 Cloud Notebook Integration

After preprocessing, the formatted dataset and its corresponding text annotations were managed using Roboflow's online pipeline. Instead of downloading gigabytes of data locally and re-uploading it, an API script was used to pull the prepared dataset directly into cloud notebooks (such as Google Colab and Kaggle) where the model training took place.

4.2 Model Training Strategy and Configuration

To find the best balance between processing speed and accuracy, three different scales of the YOLOv8 architecture were trained: YOLOv8-Small (s), YOLOv8-Large (l), and YOLOv8-Extra Large (x). Due to the high computational demands of these models, training was executed using GPU-accelerated cloud environments on Google Colab and Kaggle.

Specific hyperparameter configurations were tailored to each model size to optimize training efficiency and prevent the larger networks from simply memorizing the training data and overfitting. YOLO's inbuilt mosaic data augmentation with default settings was used along with these:

- YOLOv8-Small (s): This model was configured to train for a maximum of 160 epochs with a batch size of 64 using default baseline settings. Because smaller models have fewer parameters and are less prone to overfitting, standard configurations were

sufficient. The training automatically stopped early when the model successfully converged at 91 epochs.

- YOLOv8-Large (l): To handle the increased complexity, this model was allocated 120 epochs with a batch size of 32. A patience value of 30 was set for early stopping, and mixup of 0.2 data augmentation was introduced. Mixup blends parts of different images together, forcing the network to learn more generalized features rather than remembering the pixel patterns. The model ran for the full duration of its allocated epochs.

- YOLOv8-Extra Large (xl): As the largest and heaviest model, it was trained for 100 epochs. To maximize hardware efficiency, the batch size was set to -1, which is YOLOv8's auto-batching feature, to dynamically use the maximum possible memory. To prevent overfitting, regularization was applied: patience was reduced to 20, weight decay was set to 0.001 to penalize large weights, and augmentations of mixup = 0.2 and copy_paste = 0.1 were used. This model ran for its full epoch duration as well.

4.3 Local Inference and Evaluation Setup

4.3.1 Hardware and Software Environment

To evaluate real-time deployment capabilities on a consumer-grade device, local inference was executed on a standard laptop configuration. The specific environment is as follows:

- Hardware: Intel Core i7-10870H CPU alongside an NVIDIA GeForce RTX 3070 Laptop GPU featuring 8 GB (8192MiB) of dedicated VRAM.

- Software Environment: Python 3.10.20, PyTorch 2.11.0 with CUDA 12.8 acceleration (CUDA:0), running on the Ultralytics 8.4.56 framework. All local processing tasks were routed through the GPU for accelerated parallel computing via CUDA cores.

4.3.2 Evaluation Process and Metrics

For a comprehensive assessment of both accuracy and speed, a set of metrics was collected:

- Accuracy: Overall accuracy, Precision, Recall, mAP@0.5, and mAP@0.5:0.95 were collected to measure how reliably the models detected objects and how tightly their bounding boxes fit target classes in the dataset.

- Speed: Processing speed was tracked using FPS (Frames Per Second) to determine if the models met real-time performance thresholds (above 30 FPS).

4.3.3 Core Operational Insight

During testing, a notable performance divergence was observed. The models achieved significantly higher FPS rates when processing continuous video streams compared to inference on the static image test set. This behavior is caused by system architecture rather than model performance. When evaluating on an image test set, there is a computational bottleneck of I/O (Input/Output) overhead. The computer must repeatedly open, read, decode, and close thousands of individual image files from the storage drive one by one. Conversely, when processing a video stream, the system opens a single file or camera feed and streams a continuous sequence of frames directly into the GPU memory. This reduces the file-handling latency by a lot, allowing the GPU to run at maximum efficiency and demonstrating true real-time operational speeds.

CHAPTER 5
SCHEDULE, TASKS AND MILESTONES

5. 1 Project Task Breakdown

To successfully implement the real-time YOLOv8 object detection pipeline, the project was broken down into five core phases:

- Phase 1: Research & Environment Setup – Literature review of object detection models, YOLO models and their evolution, installing Python/CUDA environments, and setting up cloud notebooks.

- Phase 2: Data Engineering – Downloading the IDD dataset, filtering sequences to prevent data leakage, and preprocessing in Roboflow.

- Phase 3: Model Training – Configuring and training the YOLOv8-s, YOLOv8-l, and YOLOv8-x architectures on cloud GPUs.

- Phase 4: Local Deployment & Testing – Setting up local inference on the RTX 3070 laptop, running benchmarks, and testing video streams vs. static images.

- Phase 5: Evaluation & Documentation – Analyzing accuracy and inference speed metrics, compiling results, and writing the final project report.

5.2 Project Milestones and Timeline

Table 5.1 Project Milestones with Timeline

Phase	Milestone	Duration	Outcome
Phase 1	Environment Validation	4 weeks	Selected YOLOv8 Set up PyTorch and Ultralytics successfully
Phase 2	Dataset Finalization	2 weeks	Raw IDD data partitioned Used Roboflow to export intoYOLOv8 format
Phase 3	Model Training Completion	3 weeks	Trained the models YOLOv8 (s, l and x) Obtained weight files (.pt)
Phase 4	Local Inference	1 week	Inference on on local RTX 3070 device performed
Phase 5	Performance Benchmarking and Documenting	2 weeks	Data collected for FPS vs. mAP for all model scales

CHAPTER 6
PROJECT DEMONSTRATION

6.1 Verification and Evaluation Setup

The verification phase was conducted locally using Jupyter Notebook. Strictly unseen data in Test split was used. This allows for collecting real-world performance observations. Validation data was done using YOLOv8's validation function (model.val()). This function processed the test split to compute the mathematical metrics:

- Precision, Recall, mAP @ .5, mAP @ .5:.95
- Inference Time (ms), Total Pipeline (ms)

Inference time measures the time taken by the model to perform inference. Total pipeline includes the time for preprocessing the image as well as time to draw bounding boxes along with inference time.

Table 6.1 Accuracy and Inference Speed Metrics by Model

Model	Precision	Recall	mAP @ .5	mAP @ .5:.95	Inference Time (ms)	Total Pipeline (ms)
Small	0.771980	0.500772	0.564447	0.36036	4.698027	7.617228
Large	0.804556	0.571550	0.639193	0.41400	18.427985	20.994405
Extra Large	0.807774	0.554685	0.627383	0.41002	32.429785	34.869858

Matplotlib library (pyplot) was used to transform this data into graphical representation. Accuracy metrics and inference speed comparisons across the Small, Large, and Extra-Large models:

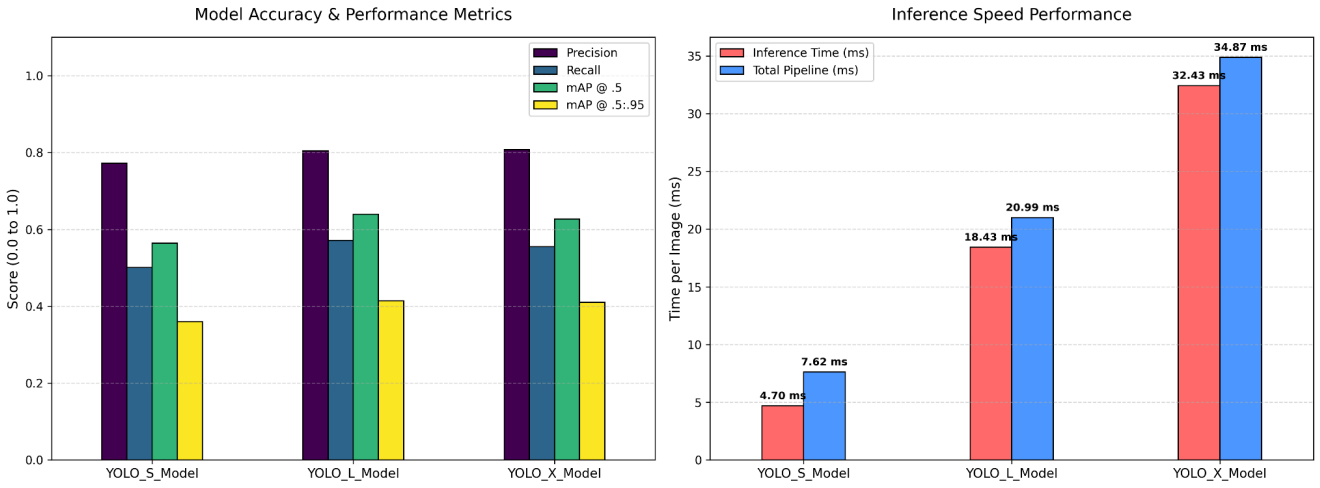


Fig. 6.1 Model Accuracy & Performance Metrics and Inference Speed Performance

The normalized confusion matrices of the models on test split data are as follows:

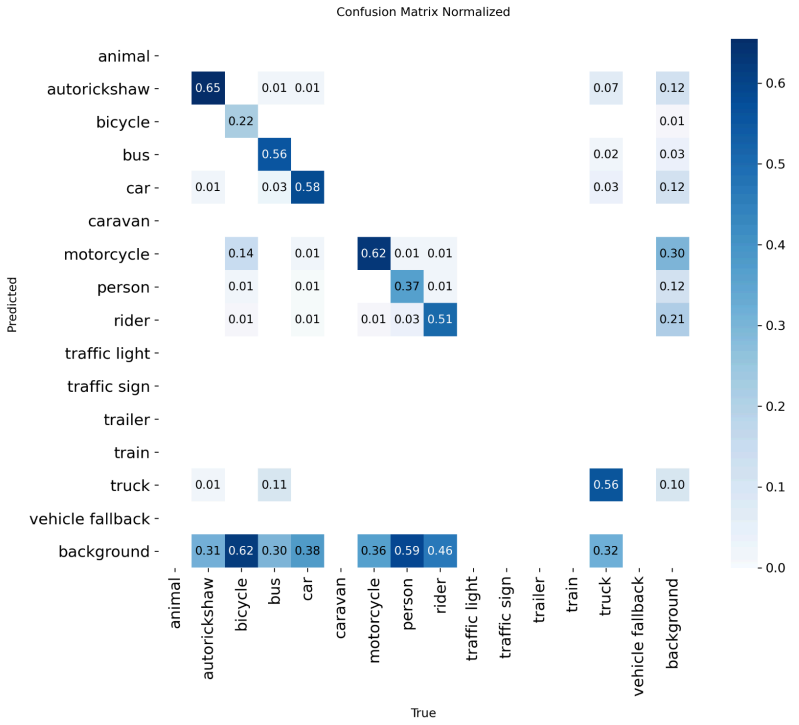


Fig 6.2 Normalized Confusion Matrix of YOLOv8-Small on Test Data

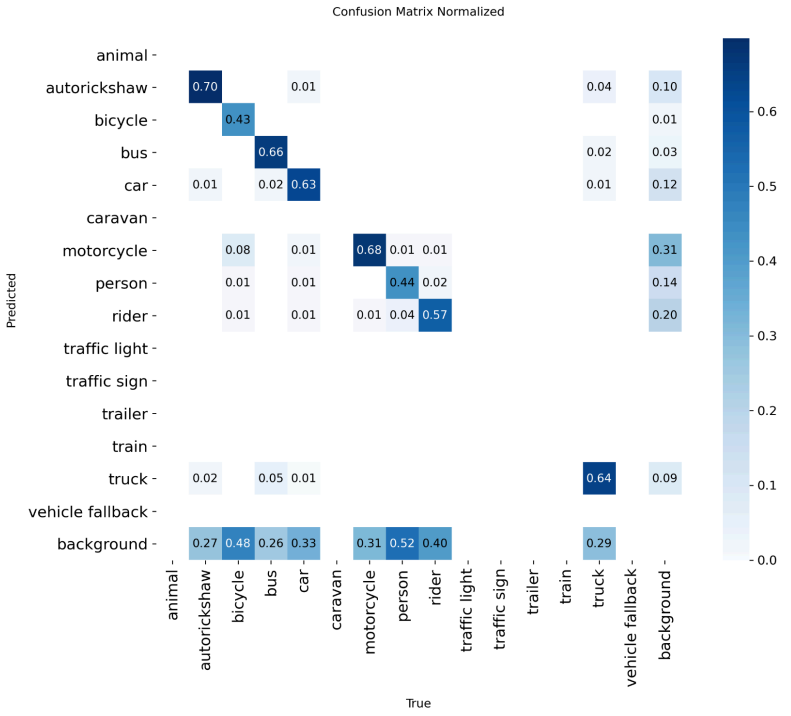


Fig 6.3 Normalized Confusion Matrix of YOLOv8-Large on Test Data

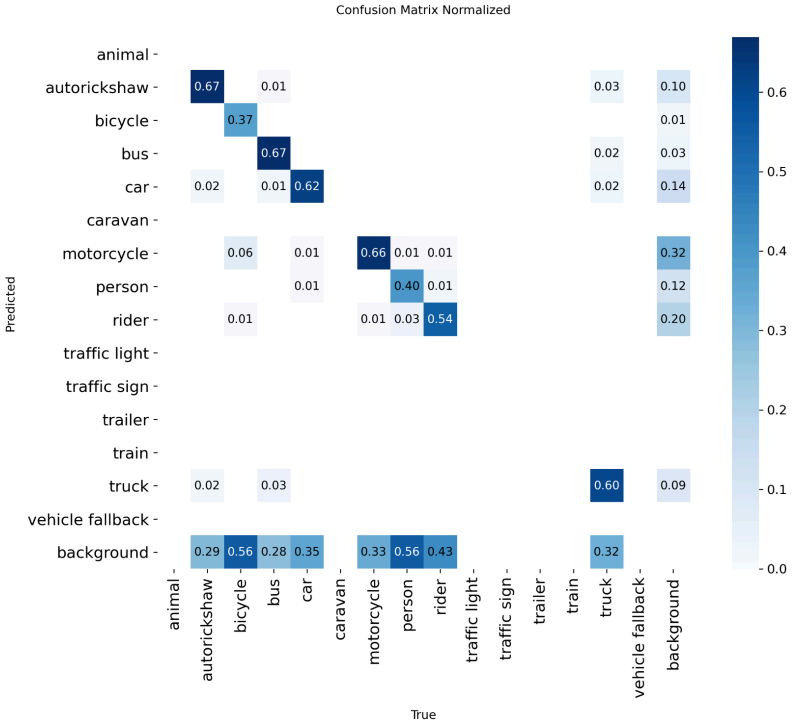


Fig 6.4 Normalized Confusion Matrix of YOLOv8-Extra Large on Test Data

6.2 Custom Inference Pipeline and Visual Overlay

The built-in YOLO prediction tool works on batch processing. To view a single image with bounding boxes drawn, a custom pipeline was built using OpenCV to give more control over how it looks. The custom script handles the images and draws a cleaner, more informative user interface onto it:

- Custom Bounding Boxes: It extracts the raw coordinate outputs from the model and uses OpenCV to draw custom bounding boxes around vehicles, people, and riders.

- Metadata Overlay: Using OpenCV, the model's name (e.g., YOLOv8-xl) and the exact inference latency (in milliseconds) are drawn directly into the top corner of the image.

Predicted images with bounding boxes, confidence scores and with model name and inference time printed on top left:

- YOLOv8 Small:

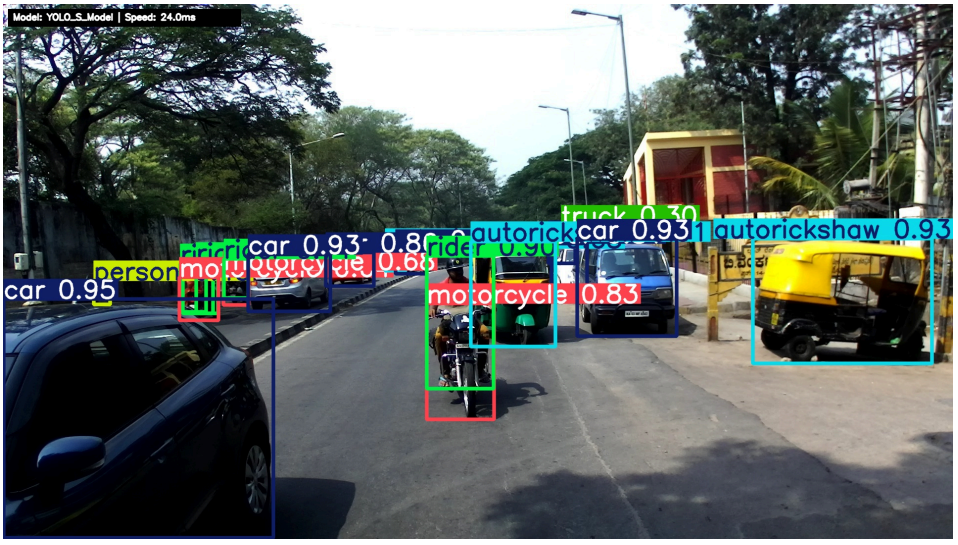


Fig 6.5 YOLOv8 Small Model Prediction-1



Fig 6.6 YOLOv8 Small Model Prediction-2

- YOLOv8 Large:



Fig 6.7 YOLOv8 Large Model Prediction-1



Fig 6.8 YOLOv8 Large Model Prediction-2

- YOLOv8 Extra-Large:



Fig 6.9 YOLOv8 Extra-Large Model Prediction-1

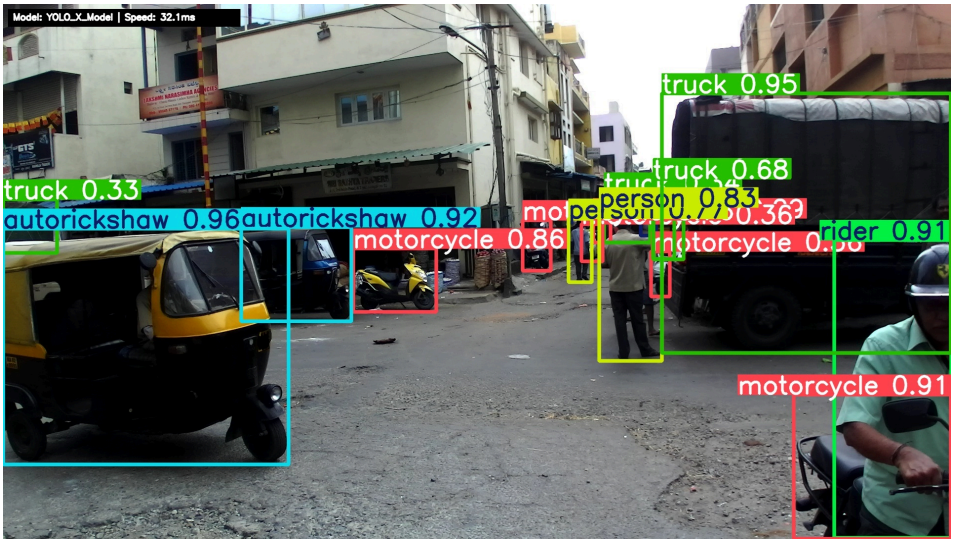


Fig 6.10 YOLOv8 Extra-Large Model Prediction-2

By looking at the images above, we can see that all three models performed well on localised vehicles like autorickshaws, trucks, down to the closely packed riders on motorcycles and pedestrians. The models are able to make these predictions successfully in real-time on a consumer-grade computer with a standard GPU.

Two-stage detectors like Faster R-CNN are known for their high accuracy, but they are slow as they use separate neural network passes to propose region zones and then classify them. A study shows that Faster R-CNN averages a latency of 170ms per frame[9]. At that speed, the system can only process about 5 to 6 frames per second (FPS). In comparison YOLOv8 looks at the entire image in a single forward pass and it manages to compress the processing time significantly. This 35ms latency corresponds to 30 FPS. These demonstration results prove that for real-time applications on standard consumer devices, the YOLOv8 architecture is superior to traditional two-stage alternatives.

CHAPTER 7
PERFORMANCE ANALYSIS AND DISCUSSION

7.1 Analysis of YOLOv8 Sizes

Changing the size of the YOLOv8 network impacts accuracy and speed. When moving from the Small (s) to the Large (l) variant, a significant jump in accuracy metrics mainly mAP @ .5 and mAP @ .5:.95 was observed in the experimental results. Total pipeline time went from 7.6ms to 21ms as well.

Meanwhile, upgrading from Large to Extra-Large did not increase the accuracy metrics, despite the model size increasing significantly. This indicates that the Large model was already sufficient to capture the complexities of the dataset. The frame latency jumped significantly from 21ms to 35ms. This extra 15ms represents the increased model complexity for virtually zero real-world accuracy gain.

YOLOv8-Large emerges as the optimal architecture for real world deployment.. While the Extra-Large model still remains within the real-time processing boundary of 35ms, it adds unnecessary complexity and demands more performance from the GPU.

7.2 Single-Stage Architecture vs. Two-Stage

Traditional two-stage networks like Faster R-CNN utilize a separate Region Proposal Network (RPN) to isolate objects before classifying them[11]. While they are precise, this design introduces a bottleneck. Because YOLOv8 calculates class scores and bounding box coordinates simultaneously through a unified regression pass, it eliminates regional generation steps[12]. This reduces processing latency down to 35ms for the Extra-Large variant. In practical traffic safety systems, this latency gap represents the difference between a system processing in real-time on standard hardware versus requiring costly cloud-based hardware.

7.3 Domain Challenges on the IDD Dataset

This project was able to prove that a model architecture originally trained on western roads could be adapted to work on Indian traffic data. Our trained models did a fantastic job of

picking up highly localized features that it had never seen before such as autorickshaws and tightly packed motorcycles.

While YOLOv8 is faster, the final metrics reveal that when compared to heavier two-stage models like Faster R-CNN, the accuracy scores are lower. This is expected, as Faster R-CNN uses a separate step to crop potential objects before guessing what they are. YOLOv8 skips that step to get a speed advantage. So it forgoes a bit of precision in exchange for running in real-time.

7.4 Limitations of the Model

When observing the test data, the model successfully classified clear vehicles, but when classifying objects with poor image quality, it ran into trouble. When dealing with tiny, far away objects and blurred out objects, the model usually missed it. Even though a human was able to recognise that object based on context and labeled it as such, the model failed to recognise them.

CHAPTER 8
SUMMARY

The main goal of this project was to see if we could build an object detection system that can handle the environment of Indian roads using commonly available hardware. By training three different sizes of YOLOv8 (Small, Large, and Extra-Large) on the Indian Driving Dataset (IDD) and running them on an RTX 3070 laptop, we have proved that this is achievable.

The experiment gave us clear takeaways. Moving from the Small to the Large model made a big difference, observable in the jump in mAP metrics. But Large to Extra-Large gave us almost no extra accuracy. Instead, it increased the latency from 21 to 35ms. The YOLOv8 Large model turned out to be the optimal size for our dataset. It was able to capture the complexity in the dataset well while running at above 45 FPS.

Finally, when comparing the heavy Extra-Large model to two-stage networks, it is still better at real-time applications when operating on standard consumer hardware. The project successfully demonstrates that high-accuracy, real-time edge deployment is possible without requiring specialized cloud hardware.

CHAPTER 9
REFERENCES

[1] Ayoola, V. B., Osam-Nunoo, G., Umeaku, C., & Awotiwon, B. O. (2024). IoT-driven smart warehouses with computer vision for enhancing inventory accuracy and reducing discrepancies in automated systems. IRE Journals, 8(5).

[2] Cheng, Q. (2021). Application of computer vision technology in industrial automation. Journal of Physics: Conference Series, 2037(1), Article 012015. https://doi.org/10.1088/1742-6596/2037/1/012015

[3] Alzubaidi, L., Zhang, J., Humaidi, A. J., Al-Dujaili, A., Ye, J., Zhou, Q., Al-Shamma, O., Duan, Y.-Y., Fadhel, M. A., Al-Cair, M., Alam, M. A., & Farhan, L. (2021). Review of deep learning: Concepts, CNN architectures, challenges, applications, future directions. Journal of Big Data, 8, Article 53. https://doi.org/10.1186/s40537-021-00444-8

[4] Khan, A., Sohail, A., Zahoora, U., & Qureshi, A. S. (2020). A survey of the recent architectures of deep convolutional neural networks. Artificial Intelligence Review, 53(8), 5455–5516. https://doi.org/10.1007/s10462-020-09825-6

[5] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 779–788). IEEE. https://doi.org/10.1109/CVPR.2016.91

[6] Hussain, M. (2023). YOLO-v1 to YOLO-v8, the rise of YOLO and its complementary nature toward digital manufacturing and industrial defect detection. Machines, 11(7), Article 677. https://doi.org/10.3390/machines11070677

[7] Ali, M. L., & Zhang, Z. (2024). The YOLO framework: A comprehensive review of evolution, applications, and benchmarks in object detection. Computers, 13(12), Article 336. https://doi.org/10.3390/computers13120336

[8] Alahdal, N. M., Abukhodair, F., Haj Meftah, L., & Cherif, A. (2024). Real-time object detection in autonomous vehicles with YOLO. Procedia Computer Science, 246, 2792–2801. https://doi.org/10.1016/j.procs.2024.09.392

[9] Mahendrakar, T., Ekblad, A., Fischer, N., White, R. T., Wilde, M., Kish, B., & Silver, I. (2022). Performance study of YOLOv5 and Faster R-CNN for autonomous navigation around non-cooperative targets. In Proceedings of the IEEE Aerospace Conference (AERO) (pp. 1–12). IEEE. https://doi.org/10.1109/AERO53065.2022.9843673

[10] IIIT Hyderabad. (n.d.). India Driving Dataset (IDD) - Dataset details. IDD. https://idd.insaan.iiit.ac.in/dataset/details/

[11] Sumit, Bisht, S., Joshi, S., & Rana, U. (2024). Comprehensive review of R-CNN and its variant architectures. *International Research Journal on Advanced Engineering Hub (IRJAEH)*, *2*(4), 959–966. https://doi.org/10.47392/IRJAEH.2024.0134

[12] Maity, M., Banerjee, S., & Chaudhuri, S. S. (2021). Faster R-CNN and YOLO based vehicle detection: A survey. In *Proceedings of the 5th International Conference on Computing Methodologies and Communication (ICCMC 2021)* (pp. 1442–1447). IEEE. https://doi.org/10.1109/ICCMC51019.2021.9418274

APPENDIX A

Python Code for Training Various Models

YOLOv8 Small:

```
from ultralytics import YOLO

model = YOLO('yolov8s.pt')
results = model.train(
    data="/content/Full-dataset/data.yaml",
    epochs=160,
    imgsz=640,
    batch=64,
    workers=2,
    amp=True,
    patience=30,
    classes=[1, 2, 3, 4, 6, 7, 8, 13],
      project="/content/drive/MyDrive/Colab
Notebooks/YOLOv8_Project/training_runs",
    name="yolo_s_160epochs"
)
```

YOLOv8 Large:

```
from ultralytics import YOLO



model = YOLO('yolov8l.pt')

results = model.train(
    data=r"/kaggle/working/Full-dataset/data.yaml",
    epochs=120,
    imgsz=640,
    batch=32,
    workers=2,
    device=[0, 1],
    amp=True,
    patience=30,
    mixup=0.2,
    classes=[1, 2, 3, 4, 6, 7, 8, 13],
```

```
    project=r"/kaggle/working/Models",
    name="l_train_120epochs"
)
```

YOLOv8 Extra-Large:

```
from ultralytics import YOLO



model = YOLO('yolov8x.pt')

#results = model.train(
    data=r"/kaggle/working/Full-dataset/data.yaml",
    epochs=100,
    imgsz=640,
    batch=-1,
    workers=2,
    device=[0, 1],
    amp=True,
    cache=False,
    patience=20,
    weight_decay=0.001,
    mixup=0.2,
    copy_paste=0.1,
    classes=[1, 2, 3, 4, 6, 7, 8, 13],
    project=r"/kaggle/working/Models",
    name="xl_train_100epochs"
)
```

APPENDIX B

Python Code for Extracting Metrics

Save Testing Results:

```
from ultralytics import YOLO

model_paths = {
   "YOLO_S_Model": r"/Project/Testing/Models/S_best.pt",
    "YOLO_L_Model": r"/Project/Testing/Models/L_best.pt",
    "YOLO_X_Model": r"/Project/Testing/Models/X_best.pt"
}
dataset_yaml = r"/Project/Testing/Dataset/data.yaml"


benchmarks = {}

for name, path in model_paths.items():
    print(f"Validating {name}...")
    model = YOLO(path)

    benchmarks[name] = model.val(
        data=dataset_yaml,
        split='test',
        save=True,
        project=r"/Project/Testing/Results",
        verbose=False
    )
```

Visualise the Metrics using matplotlib.pyplot:

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

extracted_data = []

for model_name, val_results in benchmarks.items():
    metrics = val_results.results_dict

    extracted_data.append({
        "Model": model_name,
```

```
        "Precision": metrics.get('metrics/precision(B)', 0),
        "Recall": metrics.get('metrics/recall(B)', 0),
        "mAP @ .5": metrics.get('metrics/mAP50(B)', 0),
        "mAP @ .5:.95": metrics.get('metrics/mAP50-95(B)', 0),
        "Inference Time (ms)": val_results.speed.get('inference', 0),
        "Total Pipeline (ms)": sum(val_results.speed.values())
    })

df_plot = pd.DataFrame(extracted_data).set_index("Model")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

df_plot[["Precision", "Recall", "mAP @ .5", "mAP @ .5:.95"]].plot(
    kind='bar',
    ax=ax1,
    colormap='viridis',
    edgecolor='black'
)

ax1.set_title("Model Accuracy & Performance Metrics", fontsize=14, pad=15)
ax1.set_ylabel("Score (0.0 to 1.0)", fontsize=12)
ax1.set_xlabel("", fontsize=12)
ax1.set_ylim(0, 1.1)
ax1.grid(axis='y', linestyle='--', alpha=0.5)
ax1.set_xticklabels(df_plot.index, rotation=0, fontsize=11)
ax1.legend(loc='upper right', frameon=True)

df_plot[["Inference Time (ms)", "Total Pipeline (ms)"]].plot(
    kind='bar',
    ax=ax2,
    color=['#ff6b6b', '#4d96ff'],
    edgecolor='black',
    width=0.4
)

ax2.set_title("Inference Speed Performance", fontsize=14, pad=15)
ax2.set_ylabel("Time per Image (ms)", fontsize=12)
ax2.set_xlabel("", fontsize=12)
ax2.grid(axis='y', linestyle='--', alpha=0.5)
ax2.set_xticklabels(df_plot.index, rotation=0, fontsize=11)

for p in ax2.patches:
    ax2.annotate(f"{p.get_height():.2f} ms",
```

```
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center',
                xytext=(0, 8),
                textcoords='offset points',
                fontsize=9,
                fontweight='bold')

plt.tight_layout()
plt.savefig(r"/Project/Testing/Results/model_comparision.png", dpi=300)
```

APPENDIX C

Python Code for Drawing Model Name and Inference Speeds on the Image

Using OpenCV Print Detains on Image:

```
import os
import time
import cv2
from ultralytics import YOLO

# Simply paste your literal image folder path and destination target here
TEST_IMAGES_FOLDER = r"/Project/Testing/Printed Images/Clean"
OUTPUT_ROOT = r"/Project/Testing/Printed Images/With name"

MODELS_TO_RUN = {
    "YOLO_S_Model": r"/Project/Testing/Models/S_best.pt",
    "YOLO_L_Model": r"/Project/Testing/Models/L_best.pt",
    "YOLO_X_Model": r"/Project/Testing/Models/X_best.pt"
}

VALID_EXTENSIONS = ('.jpg', '.jpeg')

if not os.path.exists(TEST_IMAGES_FOLDER):
    print(f"This folder path does not exist: {TEST_IMAGES_FOLDER}")
else:
        image_files = [f for f in os.listdir(TEST_IMAGES_FOLDER) if
f.lower().endswith(VALID_EXTENSIONS)]
    print(f"Found {len(image_files)} test images.")

    for name, weights_path in MODELS_TO_RUN.items():
        if not os.path.exists(weights_path):
            print(f"This model does not exist: {name}")
            continue

        print(f"Printing name and time for {name}")
        model = YOLO(weights_path)

        model_output_dir = os.path.join(OUTPUT_ROOT, f"{name}_preds")
        os.makedirs(model_output_dir, exist_ok=True)

        for filename in image_files:
```

```
            image_path = os.path.join(TEST_IMAGES_FOLDER, filename)

            start_time = time.time()
                    results = model.predict(source=image_path, conf=0.25,
verbose=False)
            inference_time_ms = (time.time() - start_time) * 1000

            annotated_img = results[0].plot()

                                display_text = f"Model: {name} | Speed:
{inference_time_ms:.1f}ms"
              cv2.rectangle(annotated_img, (10, 10), (480, 45), (0, 0, 0),
-1)
            cv2.putText(
                annotated_img,
                display_text,
                (20, 32),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.6,
                (255, 255, 255),
                2,
                cv2.LINE_AA
            )

            cv2.imwrite(os.path.join(model_output_dir, f"pred_{filename}"),
annotated_img)

    print(f"\nLabeled images generated at: {OUTPUT_ROOT}")
```